13.6 Exercises

Reinforcement

R-13.1 List the prefixes of the string $P =$"aaabbaaa" that are also suffixes of P.

R-13.2 What is the longest (proper) prefix of the string "cgtacgttcgtacg" that is also a suffix of this string?

R-13.3 Draw a figure illustrating the comparisons done by brute-force pattern matching for the text "aaabaadaabaaa" and pattern "aabaaa".

R-13.4 Repeat the previous problem for the Boyer-Moore algorithm, not counting the comparisons made to compute the last(c) function.

R-13.5 Repeat Exercise R-13.3 for the Knuth-Morris-Pratt algorithm, not counting the comparisons made to compute the failure function.

R-13.6 Compute a map representing the last function used in the Boyer-Moore pattern-matching algorithm for characters in the pattern string:

"the quick brown fox jumped over a lazy cat".

R-13.7 Compute a table representing the Knuth-Morris-Pratt failure function for the pattern string "cgtacgttcgtac".

R-13.8 Draw a standard trie for the following set of strings:

{ abab, baba, ccccc, bbaaaa, caa, bbaacc, cbcc, cbca }.

R-13.9 Draw a compressed trie for the strings given in the previous problem.

R-13.10 Draw the compact representation of the suffix trie for the string:

"minimize minime".

R-13.11 Draw the frequency array and Huffman tree for the following string:

"dogs do not spot hot pots or cats".

R-13.12 What is the best way to multiply a chain of matrices with dimensions that are 10×5, 5×2, 2×20, 20×12, 12×4, and 4×60? Show your work.

R-13.13 In Figure 13.14, we illustrate that GTTTAA is a longest common subsequence for the given strings X and Y. However, that answer is not unique. Give another common subsequence of X and Y having length six.

R-13.14 Show the longest common subsequence array L for the two strings:

X = "skullandbones"
Y = "lullabybabies"

What is a longest common subsequence between these strings?

Creativity

C-13.15 Describe an example of a text T of length n and a pattern P of length m such that the brute-force pattern-matching algorithm achieves a running time that is $\Omega(nm)$.

C-13.16 Adapt the brute-force pattern-matching algorithm so as to implement a method findLastBrute(T,P) that returns the index at which the *rightmost* occurrence of pattern P within text T, if any.

C-13.17 Redo the previous problem, adapting the Boyer-Moore pattern-matching algorithm to implement a method findLastBoyerMoore(T,P).

C-13.18 Redo Exercise C-13.16, adapting the Knuth-Morris-Pratt pattern-matching algorithm appropriately to implement a method findLastKMP(T,P).

C-13.19 Give a justification of why the computeFailKMP method (Code Fragment 13.4) runs in $O(m)$ time on a pattern of length m.

C-13.20 Let T be a text of length n, and let P be a pattern of length m. Describe an $O(n+m)$-time method for finding the longest prefix of P that is a substring of T.

C-13.21 Say that a pattern P of length m is a ***circular*** substring of a text T of length $n > m$ if P is a (normal) substring of T, or if P is equal to the concatenation of a suffix of T and a prefix of T, that is, if there is an index $0 \leq k < m$, such that $P = T[n-m+k..n-1] + T[0..k-1]$. Give an $O(n+m)$-time algorithm for determining whether P is a circular substring of T.

C-13.22 The Knuth-Morris-Pratt pattern-matching algorithm can be modified to run faster on binary strings by redefining the failure function as:

$$f(k) = \text{the largest } j < k \text{ such that } P[0..j-1]\widehat{p}_j \text{ is a suffix of } P[1..k],$$

where $\widehat{p}_j$ denotes the complement of the j^{th} bit of P. Describe how to modify the KMP algorithm to be able to take advantage of this new failure function and also give a method for computing this failure function. Show that this method makes at most n comparisons between the text and the pattern (as opposed to the $2n$ comparisons needed by the standard KMP algorithm given in Section 13.2.3).

C-13.23 Modify the simplified Boyer-Moore algorithm presented in this chapter using ideas from the KMP algorithm so that it runs in $O(n+m)$ time.

C-13.24 Let T be a text string of length n. Describe an $O(n)$-time method for finding the longest prefix of T that is a substring of the reversal of T.

C-13.25 Describe an efficient algorithm to find the longest palindrome that is a suffix of a string T of length n. Recall that a ***palindrome*** is a string that is equal to its reversal. What is the running time of your method?

C-13.26 Give an efficient algorithm for deleting a string from a standard trie and analyze its running time.

C-13.27 Give an efficient algorithm for deleting a string from a compressed trie and analyze its running time.

C-13.28 Describe an algorithm for constructing the compact representation of a suffix trie, given its noncompact representation, and analyze its running time.

C-13.29 Create a class that implements a standard trie for a set of strings. The class should have a constructor that takes a list of strings as an argument, and the class should have a method that tests whether a given string is stored in the trie.

C-13.30 Create a class that implements a compressed trie for a set of strings. The class should have a constructor that takes a list of strings as an argument, and the class should have a method that tests whether a given string is stored in the trie.

C-13.31 Create a class that implements a prefix trie for a string. The class should have a constructor that takes a string as an argument, and a method for pattern matching on the string.

C-13.32 Given a string X of length n and a string Y of length m, describe an $O(n+m)$-time algorithm for finding the longest prefix of X that is a suffix of Y.

C-13.33 Describe an efficient greedy algorithm for making change for a specified value using a minimum number of coins, assuming there are four denominations of coins (called quarters, dimes, nickels, and pennies), with values 25, 10, 5, and 1, respectively. Argue why your algorithm is correct.

C-13.34 Give an example set of denominations of coins so that a greedy change-making algorithm will not use the minimum number of coins.

C-13.35 In the ***art gallery guarding*** problem we are given a line L that represents a long hallway in an art gallery. We are also given a set $X = \{x_0, x_1, \ldots, x_{n-1}\}$ of real numbers that specify the positions of paintings in this hallway. Suppose that a single guard can protect all the paintings within distance at most 1 of his or her position (on both sides). Design an algorithm for finding a placement of guards that uses the minimum number of guards to guard all the paintings with positions in X.

C-13.36 Anna has just won a contest that allows her to take n pieces of candy out of a candy store for free. Anna is old enough to realize that some candy is expensive, while other candy is relatively cheap, costing much less. The jars of candy are numbered 0, 1, ..., $m-1$, so that jar j has n_j pieces in it, with a price of c_j per piece. Design an $O(n+m)$-time algorithm that allows Anna to maximize the value of the pieces of candy she takes for her winnings. Show that your algorithm produces the maximum value for Anna.

C-13.37 Implement a compression and decompression scheme that is based on Huffman coding.

C-13.38 Design an efficient algorithm for the matrix chain multiplication problem that outputs a fully parenthesized expression for how to multiply the matrices in the chain using the minimum number of operations.

C-13.39 A native Australian named Anatjari wishes to cross a desert carrying only a single water bottle. He has a map that marks all the watering holes along the way. Assuming he can walk k miles on one bottle of water, design an efficient algorithm for determining where Anatjari should refill his bottle in order to make as few stops as possible. Argue why your algorithm is correct.

C-13.40 Given a sequence $S = (x_0, x_1, \ldots, x_{n-1})$ of numbers, describe an $O(n^2)$-time algorithm for finding a longest subsequence $T = (x_{i_0}, x_{i_1}, \ldots, x_{i_{k-1}})$ of numbers, such that $i_j < i_{j+1}$ and $x_{i_j} > x_{i_{j+1}}$. That is, T is a longest decreasing subsequence of S.

C-13.41 Let P be a convex polygon, a ***triangulation*** of P is an addition of diagonals connecting the vertices of P so that each interior face is a triangle. The ***weight*** of a triangulation is the sum of the lengths of the diagonals. Assuming that we can compute lengths and add and compare them in constant time, give an efficient algorithm for computing a minimum-weight triangulation of P.

C-13.42 Give an efficient algorithm for determining if a pattern P is a subsequence (not substring) of a text T. What is the running time of your algorithm?

C-13.43 Define the ***edit distance*** between two strings X and Y of length n and m, respectively, to be the number of edits that it takes to change X into Y. An edit consists of a character insertion, a character deletion, or a character replacement. For example, the strings `"algorithm"` and `"rhythm"` have edit distance 6. Design an $O(nm)$-time algorithm for computing the edit distance between X and Y.

C-13.44 Write a program that takes two character strings (which could be, for example, representations of DNA strands) and computes their edit distance, based on your algorithm from the previous exercise.

C-13.45 Let X and Y be strings of length n and m, respectively. Define $B(j,k)$ to be the length of the longest common substring of the suffix $X[n-j..n-1]$ and the suffix $Y[m-k..m-1]$. Design an $O(nm)$-time algorithm for computing all the values of $B(j,k)$ for $j = 1, \ldots, n$ and $k = 1, \ldots, m$.

C-13.46 Let three integer arrays, A, B, and C, be given, each of size n. Given an arbitrary integer k, design an $O(n^2 \log n)$-time algorithm to determine if there exist numbers, a in A, b in B, and c in C, such that $k = a + b + c$.

C-13.47 Give an $O(n^2)$-time algorithm for the previous problem.

Projects

P-13.48 Perform an experimental analysis of the efficiency (number of character comparisons performed) of the brute-force and KMP pattern-matching algorithms for varying-length patterns.

P-13.49 Perform an experimental analysis of the efficiency (number of character comparisons performed) of the brute-force and Boyer-Moore pattern-matching algorithms for varying-length patterns.

P-13.50 Perform an experimental comparison of the relative speeds of the brute-force, KMP, and Boyer-Moore pattern-matching algorithms. Document the relative running times on large text documents that are then searched using varying-length patterns.

P-13.51 Experiment with the efficiency of the indexOf method of Java's String class and develop a hypothesis about which pattern-matching algorithm it uses. Describe your experiments and your conclusions.

P-13.52 A very effective pattern-matching algorithm, developed by Rabin and Karp [54], relies on the use of hashing to produce an algorithm with very good expected performance. Recall that the brute-force algorithm compares the pattern to each possible placement in the text, spending $O(m)$ time, in the worst case, for each such comparison. The premise of the Rabin-Karp algorithm is to compute a hash function, $h(\cdot)$, on the length-m pattern, and then to compute the hash function on all length-m substrings of the text. The pattern P occurs at substring, $T[j..j+m-1]$, only if $h(P)$ equals $h(T[j..j+m-1])$. If the hash values are equal, the authenticity of the match at that location must then be verified with the brute-force approach, since there is a possibility that there was a coincidental collision of hash values for distinct strings. But with a good hash function, there will be very few such false matches.

The next challenge, however, is that computing a good hash function on a length-m substring would presumably require $O(m)$ time. If we did this for each of $O(n)$ possible locations, the algorithm would be no better than the brute-force approach. The trick is to rely on the use of a ***polynomial hash code***, as originally introduced in Section 10.2.1, such as

$$(x_0 a^{m-1} + x_1 a^{m-2} + \cdots + x_{n-2} a + x_{m-1}) \bmod p$$

for a substring $(x_0, x_1, \ldots, x_{m-1})$, randomly chosen a, and large prime p. We can compute the hash value of each successive substring of the text in $O(1)$ time each, by using the following formula

$$h(T[j+1..j+m]) = (a \cdot h(T[j..j+m-1]) - x_j a^m + x_{j+m}) \bmod p.$$

Implement the Rabin-Karp algorithm and evaluate its efficiency.

P-13.53 Implement the simplified search engine described in Section 13.3.4 for the pages of a small Web site. Use all the words in the pages of the site as index terms, excluding stop words such as articles, prepositions, and pronouns.

P-13.54 Implement a search engine for the pages of a small Web site by adding a page-ranking feature to the simplified search engine described in Section 13.3.4. Your page-ranking feature should return the most relevant pages first. Use all the words in the pages of the site as index terms, excluding stop words, such as articles, prepositions, and pronouns.

P-13.55 Use the LCS algorithm to compute the best sequence alignment between some DNA strings, which you can get online from GenBank.

P-13.56 Develop a spell-checker that uses edit distance (see Exercise C-13.43) to determine which correctly spelled words are closest to a misspelling.

Chapter Notes

The KMP algorithm is described by Knuth, Morris, and Pratt in their journal article [62], and Boyer and Moore describe their algorithm in a journal article published the same year [17]. In their article, however, Knuth et al. [62] also prove that the Boyer-Moore algorithm runs in linear time. More recently, Cole [23] shows that the Boyer-Moore algorithm makes at most $3n$ character comparisons in the worst case, and this bound is tight. All of the algorithms discussed above are also discussed in the book chapter by Aho [4], albeit in a more theoretical framework, including the methods for regular-expression pattern matching. The reader interested in further study of string pattern-matching algorithms is referred to the book by Stephen [84] and the book chapters by Aho [4], and Crochemore and Lecroq [26]. The trie was invented by Morrison [74] and is discussed extensively in the classic *Sorting and Searching* book by Knuth [61]. The name "Patricia" is short for "Practical Algorithm to Retrieve Information Coded in Alphanumeric" [74]. McCreight [68] shows how to construct suffix tries in linear time. Dynamic programming was developed in the operations research community and formalized by Bellman [12].